

The Application of Convolution to Suffix to Prefix Rule for the Exact String Matching Problem

Zhong He Chen and R. C. T. Lee

Department of Computer Science and Information Engineering
National Chi Nan University, Puli, Nantou Hsien, 545, Taiwan ROC
zhchen@alg.csie.ncnu.edu.tw
rctlee@ncnu.edu.tw

Abstract

In this paper, we consider the exact string matching problem. We first point out a rule, called the suffix to prefix rule, which can be used to avoid the brute-force sliding window approach. The Backward Nondeterministic Matching algorithm, Backward Oracle algorithm and Reverse Factor algorithm all use this rule. To implement this rule, we have to find the longest suffix of text T which is equal to a prefix of pattern P . In this paper, we point out that convolution can be used to do this. As can be seen, the convolution technique is easy to understand and easy to program.

1 The Exact String Matching Problem

1.1 Background

String matching is a classical problem in computer science and can be applied to encryption, compression, DNA sequences analysis, imaging processing, and etc. Some searching engines such as yahoo, google and Wikipedia also apply the technique. There are many algorithms for solving the exact string matching problem, such as MP algorithm [9], KMP algorithm [6], Boyer-Moore algorithm [11], Smith algorithm [13], Backward Nondeterministic Matching algorithm [10], Horspool algorithm [5], Shift-And algorithm [15], and etc.

1.2 Terminology

Definition 1.1:

Match

Given a text $T=t_1t_2t_3\dots t_n$ and a pattern $P=p_1p_2p_3\dots p_m$, if there exists a location i in T such that $t_it_{i+1}\dots t_{i+m-1}$ is equal to $p_1p_2p_3\dots p_m$. We say that P has a match at location i in T .

For example, we are given a text $T=AT$
TGCCAA TGCCA CCA and a pattern P
 $=TGCCA$.

$T=AT$ TGCCAATG CCA CCA

$P=$ TGCCA

As the example shown above, P has a match at location 3 in T .

Definition 2.1:

Exact String Matching Problem

Given a text $T=t_1t_2t_3\dots t_n$ and a pattern $P=p_1p_2p_3\dots p_m$, find all the i in T such that $t_it_{i+1}\dots t_{i+m-1}$ is equal to $p_1p_2p_3\dots p_m$.

For example, we are given a text $T=AT$
TGCCAA TGCCA CCA and a pattern P
 $=TGCCA$.

$T=ATTGCCAATG$ CCA CCA

$P=$ TGCCA

$T=ATTGCCAATG$ CCA CCA

$P=$ TGCCA

In this case, P has two matches, location 3 and 9.

1.3 A brute-force algorithm

It is easy to design a brute-force algorithm to solve the exact matching problem. Suppose that we are given a text of length n and a pattern of length m . A brute-force algorithm compares all $n-m+1$ locations in text T which are possibly a match of P . A simple brute-force algorithm is shown as following.

Algorithm Brute-Force Algorithm for Exact String Matching

Input: A text $T = t_1t_2\dots t_n$ and a pattern $P = p_1p_2p_3\dots p_m$.

Output: find all the i in T such that $t_it_{i+1}\dots t_{i+m-1}$ is equal to $p_1p_2p_3\dots p_m$.

```
for  $i = 1$  to  $n - m + 1$  do
  if  $P_{1,m}$  is equal to  $T_{j,j+m-1}$  then
    Report that  $P$  appears at position  $i$  in  $T$ ;
  endif
endfor
```

The brute force algorithm essentially skips the pattern to the right, one step at each time. In other words, this straight-forward approach computes the pattern P with every substring of T .

Almost all exact string matching algorithms avoid computing pattern P with all substrings of T . In the following, we shall introduce a rule which can be used to move P more than one step to the right.

2 The Suffix to Prefix Rule

In this section, we introduce the suffix to prefix rule as following. First we give two definitions.

Definition 2.1

Given a string $X = x_1 x_2 \dots x_n$. S is called a suffix of X if $S = X i \dots n$, for $i \leftarrow \{1 \dots n\}$.

For example, $X=abc$. We can see that “abc”, “bc” and “c” are all suffixes of X .

Definition 2.2

Given a string $X = x_1 x_2 \dots x_n$. S is called a prefix of X if $S = x_1 \dots x_i$, for $i \leftarrow \{1 \dots n\}$.

For example, $X=abc$. We can see that “abc”, “ab” and “a” are all prefixes of X .

In almost every exact string matching algorithm, we open a window in T with the same size of the pattern P , as shown in Fig. 2.1. If the window exactly matches the pattern, a matching is found; otherwise, we will have to move the pattern to the right. It will be critical if we can slide the pattern to the right a large number of steps.

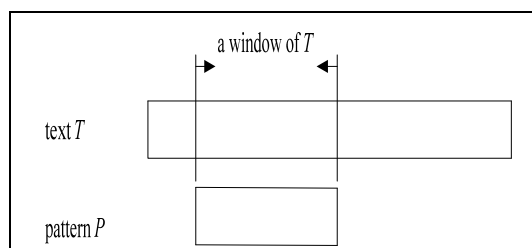


Fig. 2.1

There are many methods to determine the number of steps to slide the pattern. In this

paper, we shall use a rule, which may be called the suffix to prefix rule. Consider Fig. 2.2. Let U be the longest suffix of the window of T which is also a prefix of P . If U is also the window, an exact matching is found; otherwise, we may slide the pattern P such that the prefix of P coincides with U , as shown in Fig. 2.3.

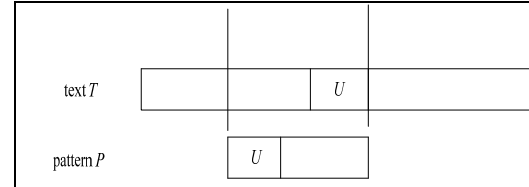


Fig. 2.2

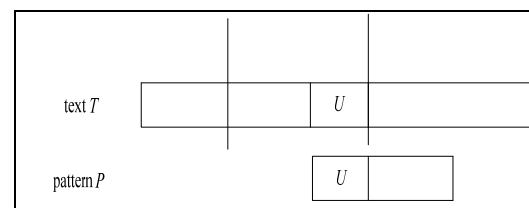


Fig. 2.3

One may ask why we may slide the pattern in such a way. Suppose that P slides smaller number of steps than that suggested by the suffix to prefix rule as shown in Fig. 2.4. As shown in Fig. 2.4, we will have a suffix U' of the window of T which is also the prefix of P where the length of U' is larger than that of U , which is impossible.

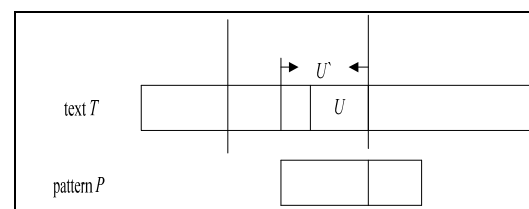


Fig. 2.4

Now, we give an example. Suppose that we are given a text $T = acbabbaccbaac$ and a pattern $P = bacc$. In Fig 2.5, the window is “acba” and U is “ba”, so we can shift the window to right by two steps.

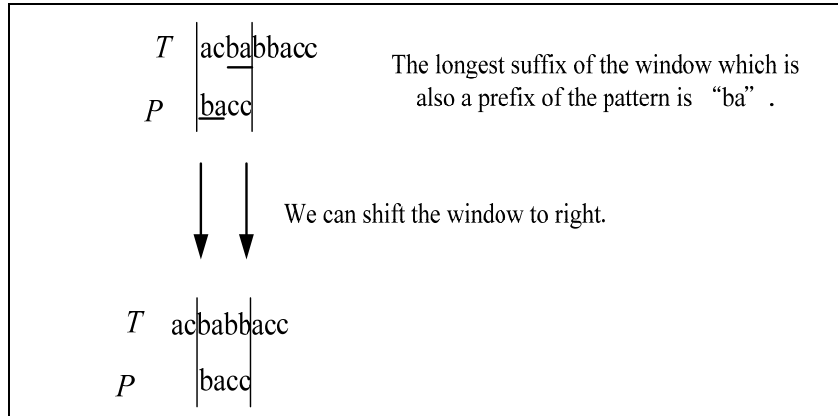


Fig 2.5

An Algorithm for suffix to prefix rule is shown as following:

Algorithm SuffixToPrefixRuleExactMatching

Input: A text T of length n and a pattern P of length m

Output: All occurrences of P in T

Step 1. Let the start position of the window of length of m be 1.

Step 2. If the start position of the window is less than $n-m+1$, go to step3; otherwise, end this algorithm.

Step 3. Find the longest suffix X of the window which is also a prefix of P and goes to step 4.

Step 4. If the length of X is m , report there is a match. Go to Step 5.

Step 5. Let the start position of the window = the start position of the window + m - length of X . Go to Step 2.

This suffix to prefix rule has been used by several researchers to design exact string matching algorithms such as the Reverse Factor algorithm [8], the Backward Nondeterministic Matching algorithm [10] and the Backward Oracle Matching algorithm [1]. The problem is how to find the longest of suffix U which is equal to a prefix of P .

The Reverse Factor algorithm uses automaton to find the longest of suffix U which is equal to a prefix of P . The Backward Nondeterministic Matching algorithm uses bit-parallel to solve the problem and the Backward Oracle algorithm uses a data structure called "oracle" to solve the problem.

In this paper, we shall show that convolution can be easily used to find such a longest suffix U .

Convolution is a technique widely used in communication [2], [4], [7], [12]. For application of convolution to string matching, consult [14].

Definition 3.1 Convolution in Discrete Case

Let $X = x_1 x_2 \dots x_m$ and $Y = y_1 y_2 \dots y_n$. Let $\otimes$ and $\oplus$ be two functions. The convolution of

X and Y is $Z = z_0, z_1, \dots, z_{m+n}$ where $z_k = \bigoplus_{i+j=k}$

$x_i \otimes y_j$, for $k=0, 1, \dots, m+n$.

To find the convolution of two strings X and Y , we may define $\otimes$ to be as following:

$x_i \otimes y_j = 1$ if $x_i = y_j$

$x_i \otimes y_j = 0$ if $x_i \neq y_j$

$\oplus$ is just the addition function.

Besides, we always find the convolution of X and Y where Y is the reverse of Y , as shown below:

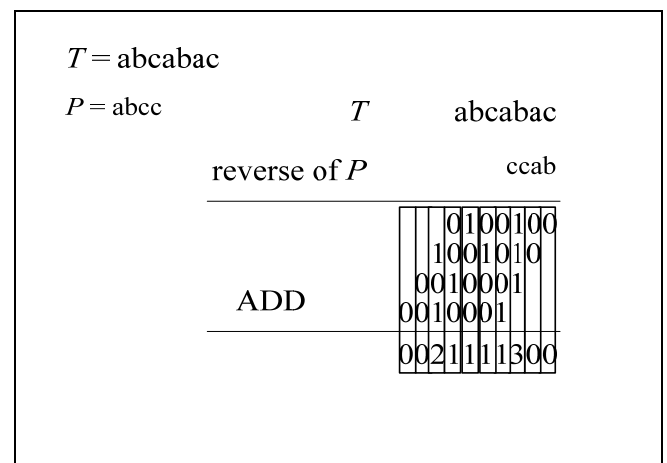


Fig3.1

Then we get the result of convolution of T and P as shown in Fig 3.2.

3 Convolution

T	abcabac
reverse of P	ccab
	0100100
	1001010
	0010001
	0010001
	0021111300

Fig 3.2

We show the meaning of the values in Fig 3.3

4 The Suffix to Prefix Rule Implemented by Convolution

In this section, we give an example to explain how to use convolution to find the longest suffix of the window of T which is also a prefix of P .

4.1 Using Convolution to Find the Longest of Suffix U Which Is a Prefix of P

Here, we give an example of convolution. Suppose that we are given a text $T = \text{cbac}$ and a pattern $P = \text{bacc}$. The convolution of T and P is shown in Fig 4.1.

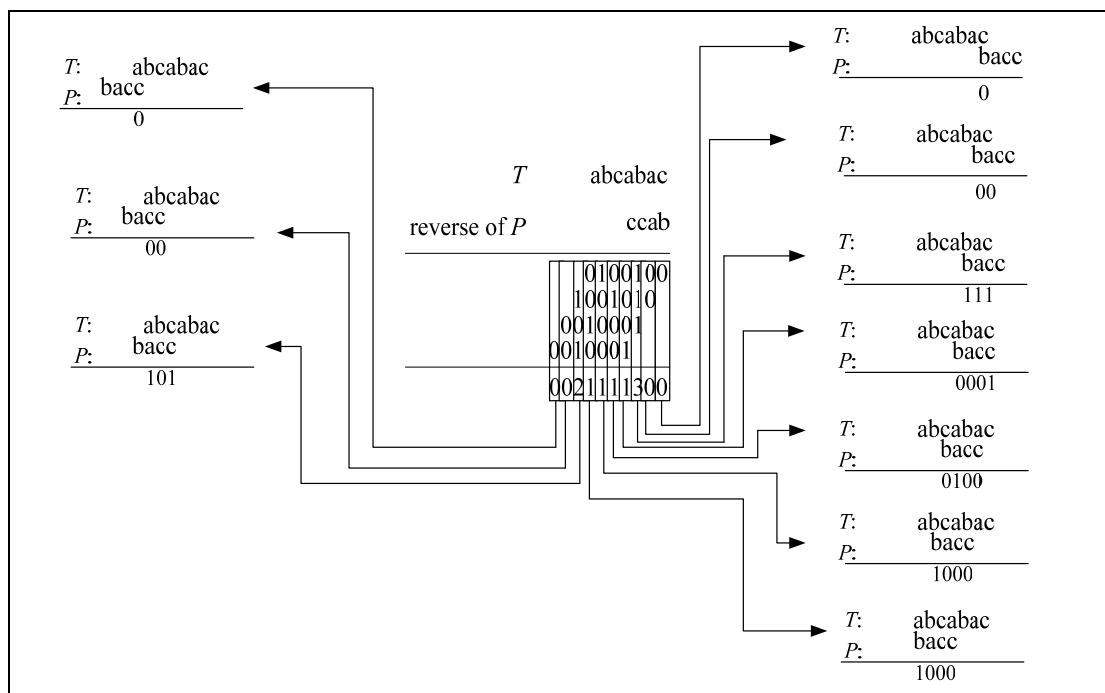


Fig 3.3

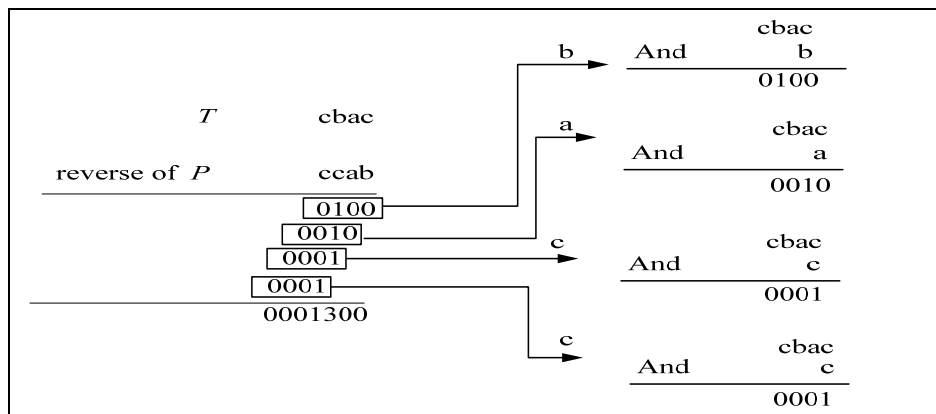


Fig 4.1

We call the lowest column “F-vector” as indicated in Fig 4.2.

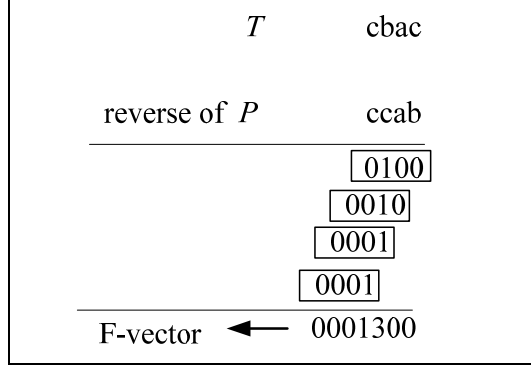


Fig 4.2

If the i -th value counted from right to left of F-vector is exactly equal to i , we know that the suffix of text T of length i is also a prefix of pattern P . From the example above, we can see that the third value of F-vector is also three as shown in Fig 4.3.

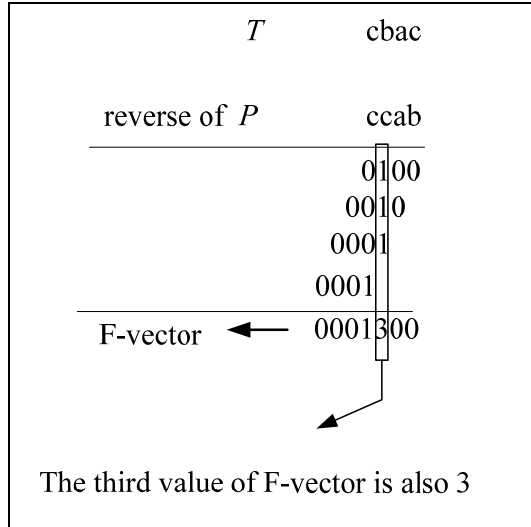


Fig 4.3

So, we know that “bac” is a suffix of T of length three which is also a prefix of pattern P . In this way, we can find out all the suffixes of T which is also a prefix of pattern P . Thus, we can also find the longest suffix of T which is also a prefix of pattern P by using convolution.

Algorithm FindTheLongestSuffixU

Input: A string $X = x_1, x_2, x_3, \dots, x_m$ and a string $Y = y_1, y_2, y_3, \dots, y_m$.

Output: The longest suffix U of X which is also a

prefix of Y

Step 1. Do a convolution of X and Y and get F-vector. Go to step 2.

Step 2. Set $i=1$ and denote the length of U be $|U|$.

Step 3. If $i \leq m$, go to step 4. Otherwise, go to step 5.

Step 4. If the i -th value counted from right to left of F-vector is exactly equal to i , Set $|U| = i$. Let $i=i+1$ and go to step 3.

Step 5. Report $U = x_{m-|U|+1}, x_{m-|U|+2}, x_{m-|U|+3}, \dots, x_m$.

4.2 The Bit-Pattern Approach

The Bit-Pattern Approach can be seen as a modified convolution. The Bit-Pattern Approach can be used to find the longest suffix of text T which is also a prefix of pattern P .

Given a string $X = x_1 x_2 x_3 \dots x_n$ and a character β , the β -bit pattern of X is defined as $b_1 b_2 b_3 \dots b_n$ where $b_i = 1$ if $x_i = \beta$ and $b_i = 0$ if otherwise.

For example, $X = \text{bacc}$.

a -bit pattern of X is 0100.

b -bit pattern of X is 1000.

c -bit pattern of X is 0011.

We give an example first. Suppose that we are given a text $T = \text{cbac}$ and a pattern $P = \text{bacc}$.

And the bit pattern of every character of the alphabet of T is shown as below:

a -bit pattern of T is 0010.

b -bit pattern of T is 0100.

c -bit pattern of T is 1001.

In the convolution of X and P is shown in Fig 4.4. We can observe that we can use bit-pattern to achieve convolution.

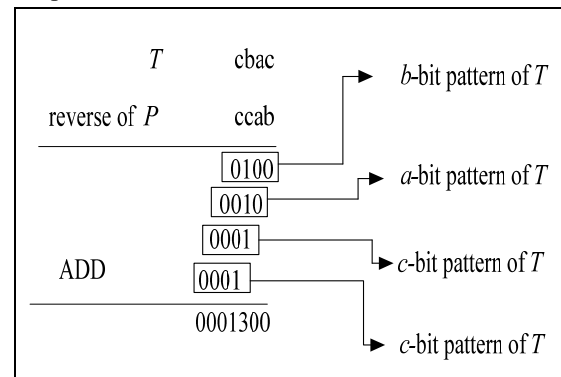


Fig 4.4

Now, we use ADD operation instead of the ADD operation above as shown above:

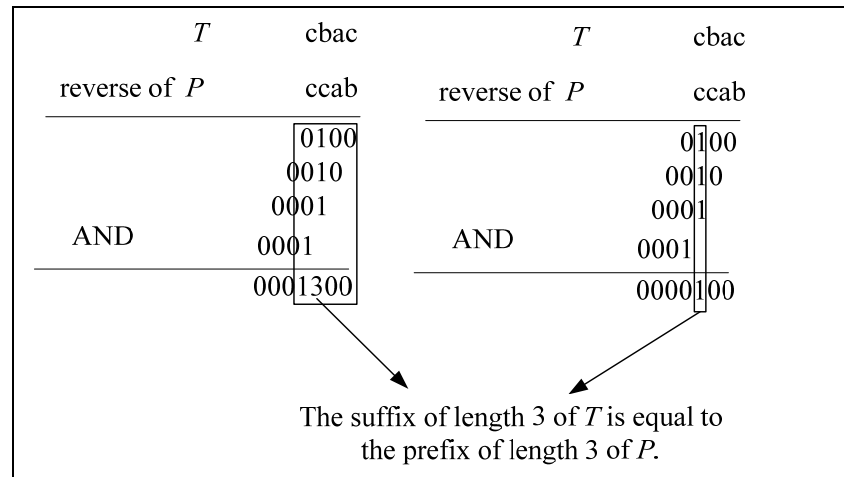


Fig 4.5

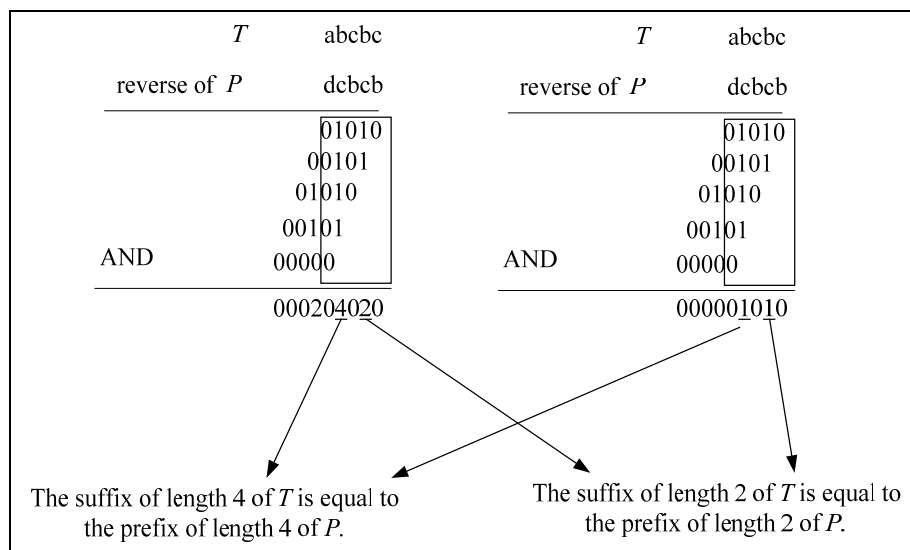


Fig 4.6

We give another example where $T = abcbc$ and $P = cbcd$ as shown in Fig. 4.6.

So, we can use AND operation which is faster than ADD operation in computers to find the longest suffix of text T which is equal to the prefix of pattern P .

Section 5 Concluding Remarks

In this paper, we propose convolution to find the longest suffix of text which is also a prefix of pattern for the suffix to prefix rule. For suffix tree and automaton both are not easy to be constructed and programmed, convolution is much easier and simpler.

References:

- [1]C.Allauzen, E.Crochemore, and M.Raffinot, Efficient experimental string matching by weak factor recognition, In Proceedings of the 12th Annual Symposium on Combinatorial Pattern Matching, number 289 in Lecture Notes in Computer, pp. 212-223, 2001
- [2]A. Bruce Carlson, Paul B. Crilly, Janet Rutledge, Communication Systems:An Introduction to Signals and Noise in Electrical Engineering, McGraw-Hill, 1986.
- [3]M.Crochemore, A. Czumaj, L.Gasieniec, S. Jarominek, T. Lecroq, W. Plandowski, and W. Rytter, Speeding up two string matching algorithms, Algorithmica, Vol. 12(4/5), 1994, 247-267.
- [4]Fischer, M.M., and Paterson, M.S., String-Matching and other products, SIAM-AMS Proceedings, Vol. 7., 1974, pp. 113-125
- [5] R.N. Horspool, Practical fast searching in strings, Software Practice and Experience, Vol. 10(6), pp.501-506, 1980

- [6] Knuth, D.E., Morris Jr, J.H., and Pratt, V.R., Fast pattern matching in strings, SIAM J.Comput., Vol. 6(1), 1977, pp. 323-350.
- [7]H. Kwakernaak and R. Sivan,Modern Signals and Systems,Prentice-Hall,1991.
- [8] T. Lecroq, A variation on the Boyer-Moore algorithm, Theoretical Computer Science, Vol. 92(1), 119-144, 1992
- [9]Morris, J. H., Jr. and Pratt, V. R., A Linear Pattern-Matching Algorithm, Report 40, University of California, Berkeley, 1970.
- [10]G. Navarro and M. Raffinot, Fast and flexible string matching by combining bit-parallelism and suffix automata, ACM Journal of Experimental Algorithmics, Vol. 5(4), 2000
- [11]R. S. and Moore, J. S. Boyer, A Fast String Searching Algorithm, Communication of the ACM, Vol. 20, 1977, pp. 762-772.
- [12]M.Selik,R.Baraniuk,Properties of Convolution,The Connexions Project and licensed under the Creative Commons Attribution License,2006
- [13]P.D. Smith, Experiments with a very fast substring search algorithm, Software—Practice and Experience, Vol. 21(10), 1991, pp.1065–1074
- [14]B.H. Wu, Convolution and Its Applications to Sequence Analysis, M.D., National Chi-Nan University, 2004
- [15]S.Wu and U.Manber, Fast text searching allowing errors, Communications of the ACM, Vol. 35(10), pp.83-91, 1992